

CONTEXTGRAPH: SELF-IMPROVING CODING AGENTS VIA CROSS-REPOSITORY EXPERIENCE GRAPHS

Anonymous authors

Paper under double-blind review

ABSTRACT

Coding agents can repeat costly debugging mistakes even when useful lessons exist in other repositories. Reusing this experience requires extracting transferable guidance and retrieving it when it can influence the next decision. We propose ContextGraph, a memory graph that organizes past coding experience into reusable strategies and warnings. The graph connects these lessons to the problems, errors, and outcomes that explain when to use them. Before generating a repair plan, the agent retrieves guidance on what to investigate and which approaches to avoid. After an error, it queries memory again with the observed failure to revise its diagnosis and next action. This allows experience from one repository to guide problem solving in another. On SWE-bench Verified and DeepSWE113, ContextGraph achieves 69.2% and 72.1% resolution, exceeding the strongest evaluated memory baselines by 2.2 and 5.9 percentage points, respectively. On Related-Lite99, its mean resolution rate is 36.3%, up 14.9 percentage points over no memory. On 20 Python tasks from LoLBench, it reaches 15.0% pass@1 versus 5.0% for all baselines, and 25.0% pass@3. Behavioral studies further examine how agents apply retrieved guidance and use failure observations to refine repairs.

1 INTRODUCTION

Each repair produces experience in locating faults, revising code, and checking behavior. Memory makes that experience useful on later tasks, offering a practical route toward recursive self-improvement (RSI): agents use past repairs to improve how they solve new problems (Yin et al., 2025; Zhang et al., 2026a).

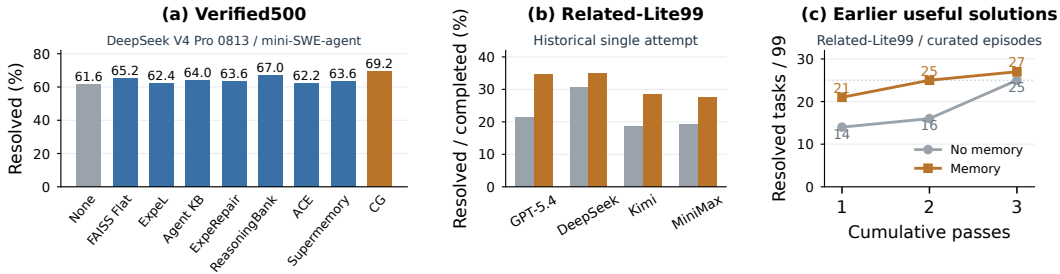


Figure 1: ContextGraph improves repair success across the evaluated settings. (a) SWE-bench Verified500 with DeepSeek V4 Pro 0813 and mini-SWE-agent; (b) Related-Lite99 with four language models; (c) cumulative resolutions on Related-Lite99 with DeepSeek V4 Pro. Grey denotes no memory and orange ContextGraph.

Recent agent-memory methods retain reflections and repair lessons (Shinn et al., 2023; Zhao et al., 2024) and organize this experience for reuse (Ouyang et al., 2026; Zhang et al., 2026b). However, experience from another repository must guide decisions in unfamiliar code. In a Django debugging case, an agent without memory spent effort on setup and broad rewrites without completing the fix. Lessons from other projects suggested isolating the failure first, tracing the error path, and avoiding a full framework setup. The history-informed agent followed this direction and repaired the bug.

The reusable lesson concerns how to investigate a problem; the agent still determines the local code change.

These observations motivate organizing experience around the decisions it can inform. We ask: *Can a memory graph turn experience from different repositories into useful guidance for planning repairs and recovering from errors on new tasks?*

Building this memory involves three challenges. First, repair trajectories contain repository-specific names, code, and setup steps. A transferable lesson must retain the useful decision and the situation in which it applies. Removing too little detail ties it to the source repository; removing too much leaves generic advice. Second, related experiences are scattered across tasks and repositories. Similar words can describe different problems, while different errors can call for the same debugging strategy. Retrieval needs the connections between problems, failed approaches, and useful actions. Third, the guidance an agent needs changes during repair. Before planning, it needs direction for investigation and warnings about costly approaches. After an error, it has new evidence that can change the diagnosis. Memory must inform both decisions using the information available at each point.

To address these challenges, we propose **ContextGraph**, a memory graph that turns experience across repositories into guidance for new repairs (Figure 3). To support transfer, *experience abstraction* extracts strategies and warnings from past trajectories, recording what to do, what to avoid, and when each lesson applies. The graph links these lessons to their source experiences, problem descriptions, and error patterns. To find useful guidance, *graph retrieval* combines semantic relevance with these connections, allowing the agent to recover related strategies across different repository contexts. To make guidance available when it can change a decision, *memory access* emphasizes two points: before generating a plan and after encountering an error. The first query uses the task description to guide investigation and approach selection. The second adds the observed error and recent attempt to guide diagnosis and revision. The agent interprets these lessons in the current codebase and carries out the edits and tests.

ContextGraph reaches 69.2% resolution on SWE-bench Verified and 72.1% on DeepSWE113, exceeding the strongest evaluated memory baselines by 2.2 and 5.9 percentage points, respectively. On Related-Lite99, its mean resolution rate is 36.3%, 14.9 points above no memory and second to Agent KB’s 38.2% (Figure 5). On 20 Python tasks from LoLBench, ContextGraph reaches 15.0% pass@1 versus 5.0% for all baselines, and 25.0% pass@3.

The contributions are summarized as follows:

- We propose ContextGraph, a memory graph for reusing coding experience across repositories. It connects abstract strategies and warnings to the problems, errors, and outcomes that explain when the guidance is useful.
- We integrate memory retrieval into planning and error recovery. Task information guides the first query; observed failures guide subsequent retrieval, helping the agent select useful actions and avoid repeating unproductive approaches.
- ContextGraph leads the evaluated methods on SWE-bench Verified, DeepSWE113, and the LoLBench Python subset, and improves Related-Lite99 by 14.9 points over no memory. Case studies show how retrieved guidance changes debugging decisions and behavioral feedback helps recover omitted requirements.

2 BACKGROUND AND MOTIVATION

Transfer a debugging decision across repositories. The Django case (Trac #33018) mishandles empty query results under negation. A failed `python-statemachine` attempt had spent its effort setting up a full Django application. Its lesson was to isolate the bug with a minimal reproduction before investing in framework setup. A successful `sievelib` trajectory also supported reproducing the failure before editing. Together, these experiences suggest an investigation strategy and warn against an expensive detour.

Apply the lesson to the current code. The history-informed agent followed the exception through query compilation to `Exists.as_sql()`, corrected its handling, and added a regression test. The no-memory attempt ended with broad formatting changes and no core fix. Memory supplied the strategy; the agent worked out the local implementation.

Figure 2 illustrates how retrieved requirements guide repair checks in our behavioral studies.

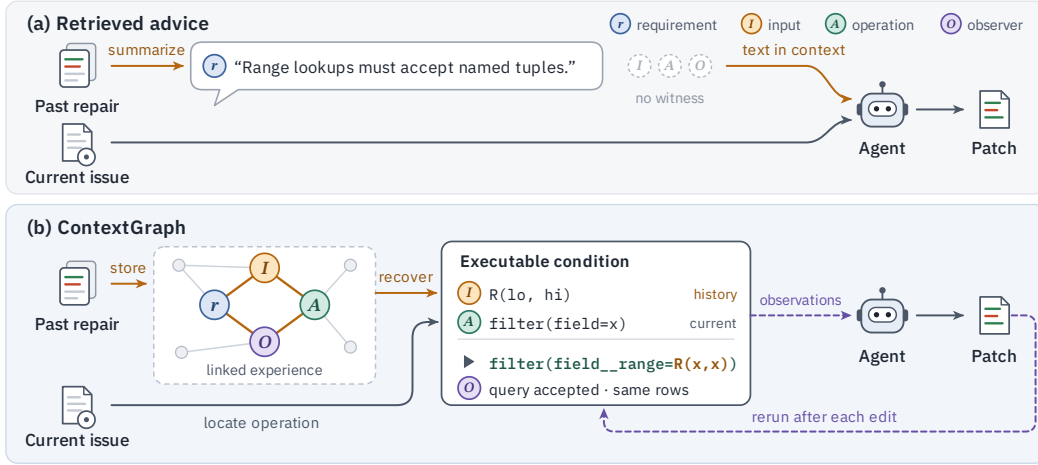


Figure 2: Behavioral study of a retrieved requirement in a Django repair. (a) The agent receives a requirement as text. (b) Linked context supports a check that combines the historical range type R with the current query. Execution feedback guides the next edit.

3 CONTEXTGRAPH: MEMORY FOR PLANNING AND ERROR RECOVERY

ContextGraph connects three designs: experience abstraction makes past decisions transferable, graph retrieval finds related lessons, and memory access brings them into planning and error recovery (Figure 3).

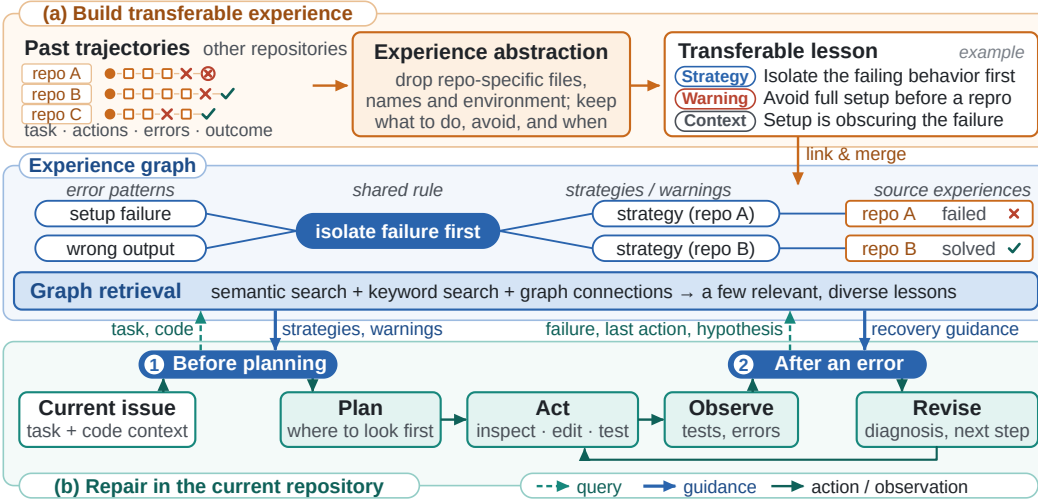


Figure 3: ContextGraph’s experience-memory architecture. (a) Past trajectories supply transferable strategies and warnings linked to source context. (b) The agent retrieves guidance before planning and after errors, then applies it through investigation, edits, and tests. The graph shows illustrative links.

3.1 EXPERIENCE ABSTRACTION: RETAIN THE USEFUL DECISION

From task descriptions, actions, errors, and outcomes, a language model extracts lessons about debugging, testing, navigation, and failed approaches. Abstraction removes source-specific names while retaining why the action helps: the failed framework setup becomes advice to isolate the bug first.

The graph links strategies to source experiences and recurring error patterns. Similar strategies share a rule, bringing together observations across repositories. Strategies describe productive actions; warnings identify unproductive approaches. Links to source outcomes help the agent interpret both.

3.2 GRAPH RETRIEVAL: FIND RELEVANT STRATEGIES AND WARNINGS

A query describes the task and the agent’s situation. Semantic search finds related lessons; keyword search matches error terms. A graph walk follows error patterns through shared rules and source strategies to past trajectories (Figure 3). Different errors can reach the same rule. The retriever merges semantic, keyword, and graph candidates, selecting a small, diverse set of lessons.

Returned memory contains actions, warnings, and source context. The agent interprets this guidance against the current code and develops the local reproduction and patch. Appendix D describes iterative agent search over the experience pool.

3.3 MEMORY ACCESS: BEFORE PLANNING AND AFTER ERRORS

Before generating a plan. The agent queries memory with the issue and available code context. Strategies suggest where to investigate, how to reproduce the problem, and what to check first; warnings identify costly approaches to avoid.

After encountering an error. A failed command, reproduction, or test supplies a more specific query: the error, recent action, relevant files, and current hypothesis. The same graph now retrieves guidance on alternative causes and next actions.

From guidance to action. The agent applies advice through repository tools: narrowing a search, tracing an exception, or adding a test. New observations guide further retrieval. Completed trajectories can enter the same abstraction pipeline, extending the experience available to later tasks. Figure 4 shows how the behavioral studies add task-specific checks to examine the application of retrieved requirements (Section 4.5).

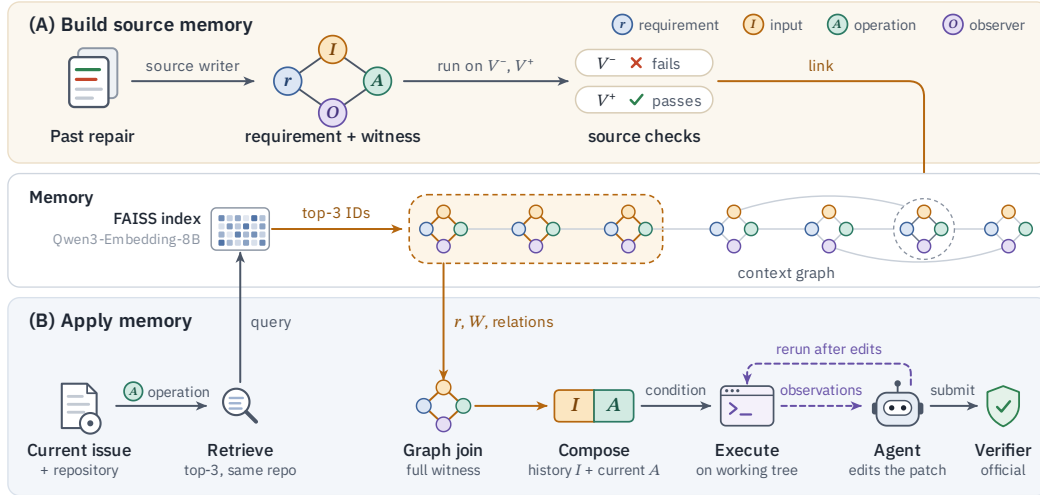


Figure 4: Workflow for the behavioral studies. (A) Historical repairs supply requirements and checked examples linked in memory. (B) Retrieval selects relevant sources; graph relations support composition with the current operation. Execution feedback guides edits before verification.

4 EXPERIMENTS

We measure task success across benchmarks, models, memory-pool sizes, and retries, then study how retrieved lessons and failure feedback guide repairs.

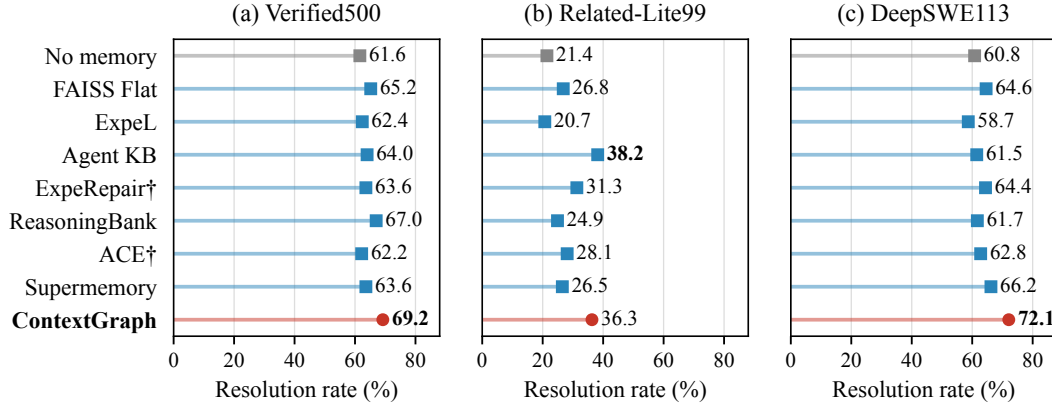


Figure 5: Repair resolution rates (%) with DeepSeek V4 Pro 0813 and mini-SWE-agent. Related-Lite99 and DeepSWE113 report five-run means. Red circles denote ContextGraph; bold values mark the best result per benchmark. † marks our adaptations to mini-SWE-agent.

4.1 SETUP AND EVALUATION

We evaluate repository-level coding tasks from SWE-bench Verified, SWE-ContextBench (Jimenez et al., 2024; Zhu et al., 2026), DeepSWE113, and a 20-task Python subset of LoLBench (lolbench26, 2026). A task is resolved when its patch passes the benchmark’s official tests. Each comparison fixes its model, memory pool, and attempt budget.

The behavioral studies examine how an agent applies retrieved guidance. They use DeepSeek V4 Pro at temperature zero and matched budgets, with researcher-prepared examples and two adapters for constructing combined checks. Their evaluation checks are separate from those supplied to guide repair. Appendix E gives the cases and inputs; Appendix G gives the implementation.

4.2 REPAIR EFFECTIVENESS

Figure 5 compares nine methods. ContextGraph reaches 69.2% on Verified500 and 72.1% on DeepSWE113, exceeding the strongest memory baselines, ReasoningBank and Supermemory, by 2.2 and 5.9 points. On Related-Lite99, its 36.3% mean improves on no memory and FAISS Flat by 14.9 and 9.5 points; Agent KB leads at 38.2%. **Finding:** ContextGraph improves repair success on all three benchmarks, leading the evaluated methods on Verified500 and DeepSWE113.

Long-horizon development. With Codex and GPT-5.5, we build the graph from other Python tasks and evaluate 20 LoLBench tasks. All seven memory baselines match no memory at 5.0% pass@1; ContextGraph reaches 15.0% pass@1 and 25.0% pass@3 (Appendix Table 16). **Finding:** ContextGraph leads the evaluated methods on these long-horizon development tasks.

4.3 PERFORMANCE ACROSS MODELS AND MEMORY-POOL SIZES

Benefits across language models. In the Related-Lite99 single-attempt comparison, memory methods draw from the same 300-summary pool within each model (Figure 6), including Mem0 (Chhikara et al., 2025) and LangMem (LangChain, 2025). With GPT-5.4, ContextGraph resolves 34/98 tasks, compared with 21/98 without memory. The oracle is given one known related summary, bypassing retrieval; ContextGraph can retrieve several. **Finding:** the memory architecture benefits all four agents, with the highest non-oracle resolution rate in each model comparison.

Pool size and source coverage. With the GPT-5.4 agent and retrieval procedure fixed, the 300-summary pool produces the highest resolution rate, 34.69% (Table 1). Source coverage rises from 41.3% to 60.7% as the pool grows from 50 to 300 summaries, while resolution does not improve at every intermediate step. **Finding:** larger pools broaden source coverage, and the largest tested pool yields the strongest repair result.

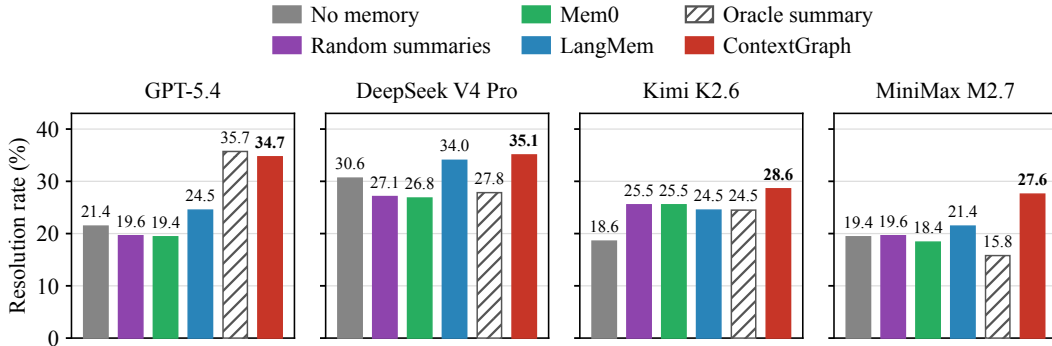


Figure 6: Related-Lite99 resolution (resolved/completed, %), one attempt per task and 300 summaries per model. ContextGraph is red; the hatched oracle receives a known related summary.

Table 1: Related-Lite99 with different memory-pool sizes, GPT-5.4, one attempt per task. Results including failure reruns appear separately in Appendix Table 8.

Number of summaries	50	100	200	300
Resolved/completed	21/96	27/98	26/97	34/98
Resolution rate	21.88%	27.55%	26.80%	34.69%
Source coverage (%)	41.3	45.8	51.4	60.7

4.4 REPAIR SUCCESS WITH REPEATED ATTEMPTS

A separate Related-Lite99 experiment uses DeepSeek V4 Pro with 500 calls and \$30 per task in both conditions. The memory contains 291 curated repair records with reference fixes, verification hints, and history preceding the target code version. Each pass is a repair round; later passes retry only unresolved tasks. ContextGraph reaches 25 resolutions in two passes; the no-memory agent needs three (Table 2). Excluding ten targets with near-duplicate source fixes retains the first-pass gain, 19/89 versus 13/89 ($p = .031$). **Finding:** useful experience helps the agent reach successful repairs with fewer rounds, with the largest gains in the first two passes.

Additional system results appear in Appendices A and C.

4.5 HOW DOES RETRIEVED GUIDANCE CHANGE REPAIR BEHAVIOR?

The Django example in Section 2 illustrates strategy transfer. Here we examine how agents apply a retrieved behavioral requirement: what context they need, which interactions they miss, and how feedback changes the next edit. Figures 2 and 4 show the checks and workflow used for these studies.

Retain the context that makes advice useful. In four paired runs, memory helps preserve query strings, remove redundant uniqueness while retaining primary keys, honor destination timezones, and preserve named-tuple ranges (Table 3). Both repairs in the first three pairs pass official verification.

We then vary the memory supplied for the constraint and timezone tasks: a written requirement, implementation details, both, or neither (Table 4). All eight repairs pass official verification. For constraints, the requirement alone leads the agent to test different columns and miss the same-column bug; the implementation retains that condition. For timezones, the requirement directs the agent to a suitable helper, whereas the old implementation leads to errors on named timezones.

On Django12663, both retrieved memory sets contain the relevant named-tuple source, yet both initial repairs miss its behavior. Preserving its constructor convention fixes named tuples while keeping ordinary subclasses working; passing every sequence’s elements as separate arguments breaks the latter (Appendix Figure 7). An exact operation filter also discards this useful source, despite its

Table 2: Related-Lite99 with curated episodic memory. Resolved counts are cumulative. Wins count tasks solved only with memory; losses count tasks solved only without it. The exact McNemar p tests whether these paired outcomes differ.

Retry budget	No memory	ContextGraph	Wins/losses	p
1 pass	14/99	21/99	7/0	.016
2 passes	16/99	25/99	9/0	.0039
3 passes	25/99	27/99	6/4	.75

Table 3: Joint current/historical behavior in complete-agent comparisons. The columns report whether repairs with and without memory preserve both the current behavior and the historical requirement.

Target	Historical addition	No memory	Memory
Django14404	Preserve query string	Fail	Pass
Django12708	Delete redundant uniqueness	Fail	Pass
Django11138	Honor destination timezone	Fail	Pass
Django12663	Preserve named-tuple range	Fail	Pass

Table 4: Memory-content interventions: one fresh attempt per input and development task. All eight pass official verification. MySQL and named-zone checks were constructed after inspecting the timezone repairs and applied to all four submissions.

Memory input	Constraint joint	Timezone joint	MySQL cases	Named-zone cases
None	Fail	Fail	Pass	Fail
Requirement	Fail	Pass	Pass	Pass
Implementation	Pass	Pass	Pass	Fail
Both	Pass	Pass	Fail	Pass

first-place similarity rank. **Finding:** advice needs its applicability context: shared columns and constructor conventions determine how these lessons transfer.

Check a historical requirement in the current workflow. A Django12663 repair passes the current lazy-User query and historical named-tuple example separately, but fails their combination. The range adapter inserts the historical type into the current query, retaining the lazy User and nested query (Figure 2).

Three agents start from the same unrepaired code, history, current program, tools, and instructions (Table 5). Separate historical checks recover both historical examples but only one of three withheld joint queries. Composed checks recover both historical examples and all three joint queries, while preserving ordinary subclasses.

Table 5: Django12663 repairs from original code with identical history, current program and failure, tools, and instructions. Two arms add composed or separate historical checks. Rows count checks, not independent tasks.

Measured behavior	Current feedback	+ Composed conditions	+ Separate history
Official target verification	Pass	Pass	Pass
Historical named-tuple examples	0/2	2/2	2/2
Withheld nonempty joint queries	2/3	3/3	1/3
Withheld ordinary subclasses	2/2	2/2	2/2
Mechanical scalar / named tuple / tuple	2/3	3/3	2/3

Timezone composition varies the database, application, and output zones. Requirement and Both pass all 72 combinations; Implementation fails 32, including 16 passed without memory. **Finding:** combining a historical trigger with the current operation exposes interactions that separate checks miss.

Use failure observations to guide the next edit. Two fresh agents revisit the failed constraint repair with the same requirement. Textual review fixes the primary-key case but misses the ordinary unique-field case; adding the historical check and its failures helps the agent distinguish constraint names and fix both. On the timezone task, further issue-based review leaves the patch unchanged, while the historical failure leads to a revision honoring the destination timezone.

Of four fresh refinements of the Django12663 candidate, history alone passes all seven measured conditions in one of two attempts; the joint program and failure do so in two of two. All four pass official verification. For the simpler redirect task, a requirement sentence already yields the same correct implementation as requirement-plus-feedback. **Finding:** targeted failure feedback helps agents act on a known requirement when textual review alone leaves the defect unresolved.

5 DISCUSSION AND RELATED WORK

Recursive self-improvement. Gödel Agent and the Darwin Gödel Machine modify agent code (Yin et al., 2025; Zhang et al., 2026a); RRSI constrains changes to the code that coordinates an agent’s tools and actions (Xia et al., 2026). AlphaEvolve uses executable evaluators to guide program evolution (Novikov et al., 2025); MulVul evaluates prompt revisions across models (Wu et al., 2026). ContextGraph contributes reusable strategies and warnings that let past repair experience guide planning and error recovery.

Learning from repair experience. Supermemory combines relational updates and hybrid retrieval (Daga et al., 2026). On SWE-ContextBench Lite99 with Claude Sonnet 4.5, no memory, Supermemory, and oracle summaries resolve 26.26%, 30.30%, and 34.34%, respectively (Zhu et al., 2026); Appendix H gives the settings. SWE-Exp and ExpeRepair distill repair experience (Chen et al., 2025; Mu et al., 2026); MemGovern and STAIR organize it as cards and plans (Wang et al., 2026; Xu et al., 2026). MemOp optimizes memory utility, ExpGraph learns retrieval, and AdaRepair-Mem selects experience for the repair stage (Guo et al., 2026; Feng et al., 2026; Luo et al., 2026). ContextGraph connects lessons across repositories and retrieves them to inform planning and error recovery.

Organizing and managing memory. MemGPT manages storage, SWE-MeM compresses coding histories, and A-MEM links memory notes (Packer et al., 2023; Gao et al., 2026; Xu et al., 2025). Zep represents temporal facts, and HippoRAG supports associative retrieval (Rasmussen et al., 2025; Gutiérrez et al., 2024). ContextGraph retrieves strategies with source context and error patterns.

Turning experience into checks. RETester adapts historical bug reports into tests for refactoring engines (Wang et al., 2025). MIRA separates reusable requirements from SQL corrections, and TRACE compiles user corrections into runtime checks (Liu et al., 2026; Zhou et al., 2026). Our development cases combine historical triggers with current workflows to expose failures missed by separate checks. The trigger and expected outcome give the agent a concrete repair objective.

6 CONCLUSION

We propose ContextGraph, an experience-graph architecture for self-improving coding agents. It turns past trajectories into transferable strategies and warnings linked to their source context. Graph retrieval supplies this guidance before planning and after errors to help agents choose and revise their approach. ContextGraph achieves 69.2% resolution on SWE-bench Verified and 72.1% on DeepSWE113, exceeding the strongest evaluated memory baselines by 2.2 and 5.9 percentage points, respectively, and raises Related-Lite99 mean resolution from 21.4% to 36.3%. On the 20-task LoL-Bench Python subset, pass@1 is 15.0% versus 5.0% for all baselines. These results establish ContextGraph as a strong starting point for coding-agent memory that learns from past repair experience.

REFERENCES

- Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2025. URL <https://arxiv.org/abs/2507.23361v2>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- Soham Daga, Sreeram Sreedhar, and Dhruva Shah. Supermemory is state of the art in agent memory, 2026. URL <https://supermemory.ai/research/longmembench/>. Supermemory Research, LongMemEval-S evaluation report, May 2026.
- Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, and Jiaxuan You. ExpGraph: Model-agnostic experience learning with graph-structured memory for LLM agents, 2026. URL <https://arxiv.org/abs/2605.30712v1>.
- Shuzheng Gao, Wenhao Zeng, Zhaojian Yu, Jianqiao Wangni, Chaozheng Wang, Kai Cai, Shilin He, and Michael R. Lyu. SWE-MeM: Learning adaptive memory management for long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2606.28434v1>. Version 1.
- Xuehang Guo, Zora Zhiruo Wang, Qingyun Wang, Graham Neubig, and Xingyao Wang. Enhancing software engineering through closed-loop memory optimization, 2026. URL <https://arxiv.org/abs/2606.05646v1>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbdl-Abstract.html.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- LangChain. LangMem, 2025. URL <https://github.com/langchain-ai/langmem>. Software library.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- lolbench26. LoLBench: Long-horizon tasks on large software systems. Hugging Face dataset, 2026. URL <https://huggingface.co/datasets/lolbench26/LoLBench>.
- Z. C. Luo, J. C. Guo, W. J. He, S. Y. Wang, J. C. Yu, F. M. Zhao, Y. Chen, T. Cao, L. Q. Liu, N. Zheng, W. Xu, J. Jiang, and Z. M. Zhao. AdaRepair-Mem: Adaptive experience orchestration for repository-level program repair, 2026. URL <https://arxiv.org/abs/2609.20130v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.

- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. Agent KB: Leveraging cross-domain experience for agentic problem solving, 2025. URL <https://arxiv.org/abs/2507.06229>.
- Haibo Wang, Zhuolin Xu, and Shin Hwei Tan. Testing refactoring engine via historical bug report driven llm, 2025. URL <https://arxiv.org/abs/2501.09879v2>.
- Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, et al. MemGovern: Enhancing code agents through learning from governed human experiences, 2026. URL <https://arxiv.org/abs/2601.06789v2>.
- Zihan Wu, Jie Xu, Yun Peng, Chun Yong Chong, and Xiaohua Jia. MulVul: Retrieval-augmented multi-agent code vulnerability detection via cross-model prompt evolution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8660–8673, 2026. doi: 10.18653/v1/2026.acl-long.391. URL <https://aclanthology.org/2026.acl-long.391/>.
- Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. RRSI: Regularized recursive self-improvement of agent harnesses, 2026. URL <https://arxiv.org/abs/2609.24972v1>. Version 1.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html.
- Yisen Xu, Jiayuan Zhou, Ruiqi Pan, and Tse-Hsun Chen. Reusing past repairs through hierarchical trajectory abstraction for coding agents, 2026. URL <https://arxiv.org/abs/2607.29658v1>. Version 1.
- Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 27890–27913, 2025. doi: 10.18653/v1/2025.acl-long.1354. URL <https://aclanthology.org/2025.acl-long.1354/>.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=pUpzQZTvGY>.

- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026b. URL <https://arxiv.org/abs/2510.04618v3>.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176v3>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024. doi: 10.1609/aaai.v38i17.29936. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29936>.
- Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents, 2026. URL <https://arxiv.org/abs/2606.13174v1>.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.

A ADDITIONAL SYSTEM COMPARISONS

These tables give additional system comparisons and their experiment settings. The earlier Verified500 run used 75 steps per task; the main text reports the later 346/500 comparison. Related-Lite results cover summary memory, episodic memory, and memory-interface comparisons.

Table 6: Historical SWE-bench Verified500, GPT-5.4, 75 steps and uncapped per-task cost. Rates use all 500 tasks, including unfinished attempts. Wins/losses and exact McNemar p compare each method with no memory.

Method	Resolved	Rate	Gain (pp)	Wins/losses	p
No memory	309	61.80%	—	—	—
ExpeL	312	62.40%	+0.6	43/40	.826
FAISS	327	65.40%	+3.6	49/31	.057
ContextGraph	318	63.60%	+1.8	50/41	.402

Memory configurations. The Verified500 system indexes trajectory fragments, rules, and communities with dense, lexical, and graph retrieval. The Related-Lite99 system indexes 300 Experience summaries with repository-filtered lookup and a memory tool; its production repository-filtered route differs from generic channel fusion. The Related-Lite99 episodic system serves reference fixes, verification hints, and pre-base history from 291 episodes at pre-plan and post-error query points. These configurations precede the Qwen executable-witness graph.

Task counts and attempts. Verified500 uses a fixed 500-task denominator; completed counts were 486 for no memory, 488 for ExpeL, 490 for FAISS, and 484 for ContextGraph. Related-Lite99 reports resolved/completed in each condition. The main-text comparison allows one attempt; a separate historical aggregate includes failure reruns. The pool-size table uses only the single-attempt results: 21/96, 27/98, 26/97, and 34/98 for 50, 100, 200, and 300 summaries, respectively. The MiniMax M2.7 campaign used a gateway-compatibility shim.

Related-Lite99 source content. The episodic campaign served static source episodes: its original online deposit path omitted fields needed for retrieval. The historical overlap analysis identified ten targets with near-duplicate source fixes (token-Jaccard at least 0.85); removing them gives an 89-task frame. DeepSeek V4 Pro retains a first-pass gain of 19/89 versus 13/89 ($p = .031$). The full-set cumulative two-pass gain of nine exceeds the campaign’s preregistered threshold of eight.

Table 7: Historical curated-episode Related-Lite99 replication, one pass per backbone. Last column gives recorded control/memory counts and paired p after removing ten near-duplicate targets; arm-specific completed denominators for that subset are not provided in the original table.

Backbone	Control	Memory	C/M only	p	Reduced set
DeepSeek V4 Pro	14/99	21/99	0/7	.016	13/19 ($p = .031$)
DeepSeek V3.2 exp	16/98	20/99	4/8	.39	15/18 ($p = .55$)
GLM 5.2	17/99	24/99	1/8	.039	15/21 ($p = .070$)
Kimi K3	29/99	31/99	3/5	.73	26/28 ($p = .73$)

Table 8: Historical Related-Lite99 GPT-5.4 aggregate including failure reruns. The original table does not report attempts per method; Figure 6 gives the separate single-attempt comparison.

Memory	Resolved/completed	Rate
Random summaries	19/97	19.59%
Mem0	19/98	19.39%
LangMem	24/98	24.49%
No memory	25/98	25.51%
Oracle summary	35/98	35.71%
ContextGraph	41/98	41.84%

Table 9: Historical Verified development subset. Rates use completed evaluations. The preprint transcribed this table from an experiment screenshot; 66.6% retains the recorded display precision.

Method	Completed/planned	Resolved	Recorded rate
ContextGraph	48/50	32	66.6%
ExpeL	50/50	32	64.0%
FAISS	49/50	30	61.2%
No memory	50/50	29	58.0%

Table 10: Historical SWE-bench-Pro50, recovered from a disabled preprint section. Memory arms share a 3,183-strategy pool and a \$10 per-task cap; no memory is uncapped. Rates use completed evaluations; costs are recorded totals under these different budgets.

Method	Completed/planned	Resolved	Rate	Cost (\$)
ContextGraph	47/50	11	23.40%	41
FAISS	47/50	5	10.64%	68
No memory	46/50	2	4.35%	1,686

Table 11: Historical trajectory-graph snapshot, using text-embedding-3-large vectors with 3,072 dimensions. The new executable-witness graph uses a separate Qwen embedding space.

Node label	Count	Role
Trajectory	1,795	Source episode
Fragment	13,813	Action and observation slice
ErrorPattern	586	Error type and keywords
Strategy	9,600	Extracted rule
CanonicalRule	7,835	Consolidated rule
PlaybookEntry	8,910	Retrieval-ready entry
Community	781	Linked-fragment summary
ProblemSummary	1,795	Episode problem abstraction
Total nodes	45,115	
Total edges	73,975	Seven typed relations

B EARLIER INTERFACES AND DEVELOPMENT COMPARISONS

Fail-to-pass tests check the requested fix; pass-to-pass tests check behavior that worked before the repair. Table 12 compares six interfaces in a Related-Lite99 experiment: a separate DeepSeek V4 Pro campaign with 500 calls and the last five observations retained. Its frozen verdict requires fail-to-pass success and does not veto every pass-to-pass regression. The 88-task frame belongs to this campaign’s source-overlap analysis.

Table 12: Earlier Related-Lite99 memory interfaces and their original 88-task sensitivity frame. Gains and paired exact tests use the full 99-task frame.

Memory	Resolved/99	Resolved/88	Gain (pp)	<i>p</i>
No memory	17	16	—	—
Flat patches	18	18	+1.01	1.000
Patches + procedure card	18	15	+1.01	1.000
Graph	12	12	-5.05	.227
Rules	13	12	-4.04	.424
File list	9	8	-8.08	.057

Table 13: Django14404 development comparison. All candidates pass the official 2 fail-to-pass and 323 pass-to-pass tests.

Candidate	Prefix	Query	Prefix + query
Initial repair	Pass	Fail	Fail
Issue-based revision	Pass	Fail	Fail
Historical patch revision	Pass	Pass	Pass
Executed relation revision	Pass	Pass	Pass

Intervention scope. Content comparisons start fresh agents from the original development target; refinement comparisons start from a common completed candidate. Additional MySQL, named-zone, and ordinary-subclass checks were constructed after inspecting the corresponding repairs, then applied across their comparison group. The composition pair holds history, current example and failure, and the execution interface fixed; the added input is the composed condition.

Operation-dependent preservation. In Matplotlib20826, clearing and replotting twin axes restores finite data bounds, so failure of an isolated historical plotting example does not describe the state after that workflow. In Django16819, canceling a same-name index remove/add pair loses a required change of indexed column. These observations motivate applying memory conditions along the current operation sequence.

Additional development outcomes. The closed DeepSWE Kea development baseline passes 9/12 fail-to-pass and 139/139 pass-to-pass tests (binary reward 0). The closed JDK5 LoLBench baseline originally submitted no artifact; a separate no-inference recovery of its recorded patch passes 0/5 fail-to-pass and 51/51 pass-to-pass tests (binary reward 0). These are development outcomes. The historical Verified500 table is a different evaluation from Verified150.

Executable-witness bank coverage. The archived 273-source bank provides at least one witnessed source from the same repository for 133/150 Verified targets, 91/99 Related-Lite targets, 2/100 LoLBench targets, and 0/113 DeepSWE targets. These counts describe the executable-witness bank used during development, rather than the cross-repository strategy memory in the main benchmark comparison.

C ADDITIONAL RECORDED EXPERIMENT RESULTS

These results complement the three-benchmark comparison in Figure 5. The Related-Lite99 and Gemini summaries below retain their supplied aggregates; detailed run settings were not included. The LoLBench experiment uses the Python subset described below. Earlier campaigns appear in Appendix A.

Table 14: Additional Related-Lite99 comparison. Entries report resolved/completed counts.

Method	Resolved/completed
No memory	21/98
Free summaries	19/98
Mem0	19/98
LangMem	24/98
ContextGraph	35/98

Table 15: New Gemini 3.1 Pro/live-SWE-agent comparison. The gain column reports relative improvements from the supplied rates.

Metric	No memory	ContextGraph	Relative gain
pass@1	82.3%	88.9%	+8.0%
pass@3	82.6%	90.4%	+9.4%
pass@5	82.8%	91.7%	+10.7%

Token efficiency. The new summary reports 236K tokens per task without memory and 201K with ContextGraph, a 14.8% reduction (approximately 15%).

LoLBench Python subset. We select 20 Python tasks from LoLBench (lolbench26, 2026), which evaluates long-horizon development from enhancement proposals in large software systems. The agent is Codex with GPT-5.5. We construct the experience graph from other Python tasks, separate from the 20 evaluation tasks.

Table 16: Success rates on 20 LoLBench Python tasks with Codex and GPT-5.5. $\text{pass}@k$ reports the fraction of tasks solved within k attempts.

Method	Metric	Success rate (%)
No memory	pass@1	5.0
FAISS Flat	pass@1	5.0
ExpeL	pass@1	5.0
Agent KB	pass@1	5.0
ExpeRepair	pass@1	5.0
ReasoningBank	pass@1	5.0
ACE	pass@1	5.0
Supermemory	pass@1	5.0
ContextGraph	pass@1	15.0
ContextGraph	pass@3	25.0

D FULL-CONTEXT CODE-AGENT RETRIEVAL

This proposed extension lets a selector explore the experience graph using the coding agent’s complete current-task context. Beyond the issue, that context includes inspected code, earlier actions, and observed failures, which can change the guidance needed for the next decision.

Let C_t contain the task and conversation, inspected code, edits, actions, and observations available at step t . The selector operates on C_t and a frozen experience pool $\mathcal{M}$. It can issue semantic or keyword queries, open source records, follow links between strategies and errors, and compare lessons from different experiences. It returns strategies and warnings together with their source IDs and an explanation of how they can help the current repair. Search can recur before a new plan or after an error.

Full context describes access to the current task; the selector explores the experience pool through search and record reading. The returned advice uses the same memory interface as Section 3, and the coding agent applies it through its own investigation, edits, and tests.

Current implementation. The repository provides context-aware candidate reranking and a selector over all 300 compact experience cards. These prototypes cap task or working-state text at 4,000 characters and limit candidate content. They provide starting points for the proposed full-context, iterative search.

E DEVELOPMENT CASES AND INPUT CONSTRUCTIONS

This appendix gives the inputs, source requirements, and repair observations for the behavioral studies in Section 4.5.

E.1 REQUIREMENT MEMORY: PRESERVING TRIGGERING CONDITIONS

The executable-analysis experiments use a Verified split of 280 source tasks, 70 development tasks, and 150 held-out evaluation tasks. The cases below are controlled changes to the memory supplied to DeepSeek V4 Pro through OpenRouter, temperature zero and matched budgets within each comparison. They measure whether history adds a required behavior to the submitted repair. Table 3 presents four such additions. Django14404 uses the first-ranked source issue and patch; Django12708 and Django11138 come from a fixed six-task group receiving three retrieved rules as plain text per task. Both conditions pass official verification on all six tasks. The historical requirements distinguish the completeness of those repairs.

An omitted behavior of the same API. Django14404 reports that administrative redirects lose a deployment prefix. Both agents fix the prefix and pass official verification, but only the repair informed by source16612 also preserves the query string. A joint input combines the prefix with repeated and percent-encoded query values, `keep=a%2Fb&keep=c%20d`. Only the history-informed repair preserves both.

A neighboring operation. Django12708 reports ambiguous deletion of an ordinary index when a unique constraint uses the same columns. The first retrieved source, 15732, instead concerns deletion of a unique constraint sharing its column with a primary key. Both repairs fix ordinary-index selection; only the memory-informed repair also excludes primary keys when deleting unique constraints. The source-generated PostgreSQL check runs unchanged on both target repairs. A separate combined input creates an ordinary index, redundant uniqueness, and a primary key on the same column. Both repairs preserve ordinary-index deletion, but only the memory-informed repair also deletes redundant uniqueness while retaining the primary key and data. History supplies a requirement for the neighboring operation.

An independent argument of the same operation. Django11138 requires date conversions to respect the database’s source timezone. Source13363 requires `TruncDate` and `TruncTime` to honor an expression’s explicit destination timezone. Both agents repair the source timezone, but only the memory-informed repair also honors the destination. A source-generated check distinguishes the historical versions and runs unchanged on the target repairs. On an independent SQLite input combining a UTC+1 database with an explicit New York destination, only the memory-informed repair returns the correct date and time. The issue and history constrain different ends of the conversion.

A historical input type in the current query. A fourth pair repairs Django12663 from its original code, comparing no memory with ContextGraph using source13590. Both agents receive the same public current program, tools, backbone, and repair budget; only the memory arm receives the historical requirement and its checks. Each arm completes one repair. The joint checks combine the current lazy User with a named-tuple range in nonempty queries. The no-memory repair fails these checks, while the memory-informed repair passes.

A requirement can suffice without the source implementation. On the redirect task, retrieved text memory and its original issue/patch replacement produce identical repairs preserving the joint behavior. Two new agents then revise the ordinary patch with a source-generated requirement: the redirect must include the original query string. One also receives the check and its failed observations. Both submit identical repairs passing official and joint checks. The sentence-only agent finds `get_full_path()` in the current repository: memory supplies the behavior, and the agent locates its implementation.

Which information makes the requirement usable? We select the constraint and timezone benefit cases and run four fresh complete agents per task, fixing the single historical source and its delivery before editing. Using only the historical task, we construct a written requirement and an

	Official verification	Historical named tuples (2)	Named tuple + lazy User (1)	ADDITIONAL DIAGNOSTIC Ordinary tuple/list subclasses (2)
Memory supplied for revision				
Original issue + patch	Pass	Pass	Pass	Pass
Requirement sentence	Pass	Fail	Fail	Pass
Requirement + check + execution failures	Pass	Pass	Pass	Fail

Figure 7: Memory content changes which behaviors a repair preserves. One revision per input starts from the same Django12663 candidate and source13590. Parentheses give case counts; each cell reports all cases in its group. The ordinary-subclass inputs were chosen after inspecting the feedback repair. All three repairs pass official verification, while only original history passes every displayed behavior.

implementation reference; the combined input concatenates them unchanged. All eight repairs pass official verification (Table 4). On Django12708, only Implementation and Both also remove redundant uniqueness while preserving the primary key. The Requirement agent tests a field with `unique=True`, but places `unique_together` on a different, two-column set. This nearby passing test loses the same-column condition stated in memory. On Django11138, all three memory inputs preserve the explicit destination timezone, while the fresh no-memory repair omits it. The Requirement agent finds `self.get_tzname()` in the target implementation without being given that helper in memory.

The timezone repairs also differ on further conditions. The repair given Both still generates an unnecessary MySQL conversion between identical zones. The repair given Implementation instead misinterprets a named Paris source timezone, producing New York time 19:21 instead of 17:30 on a summer input. The repair given Requirement passes both additional diagnostics. These concrete omissions distinguish carrying a requirement, applying its triggering condition, and implementing the resulting behavior correctly. Two subsequent agents start from the original unrepaired code and receive the same constraint requirement, with or without an example validated on the historical repair; both pass official and joint checks. The same requirement succeeds when its trigger is reconstructed correctly; preserving that input construction is a useful role for the graph.

Preserve the input relation. On Django12663, both a set of three code-connected sources and the ten most similar sources contain the historical named-tuple repair, source13590, yet neither initial repair recovers its behavior. Three fresh revisions start from the same candidate and source (Figure 7). Original history and execution feedback recover named tuples; the requirement sentence alone does not. The feedback repair expands every non-builtin sequence into positional arguments, which also breaks ordinary tuple and list subclasses. Original history retains the constructor distinction and passes all displayed behaviors. The graph therefore needs to store how the input type is constructed, alongside its endpoint values.

Retain candidates that can supply an omitted condition. In this same case, demanding an operation match from the initial public-code localization excludes source13590, which FAISS ranks first. Keeping the similarity candidate lets the graph return its input relation and construct the joint program described next. This motivates the division in Figure 4: FAISS proposes sources; graph relations support their adaptation to the current operation. The usefulness of a candidate is established by what its condition adds to the repair.

E.2 CONDITION COMPOSITION: CHECKING INTERACTIONS

Join the historical input to the current operation. The current issue involves a lazy User in a nested query; history contributes a named-tuple range. On a separate history-informed repair, the current scalar query and the historical range query each pass, while a query combining them fails.

The combined check reveals that the patch handles one reconstruction of the input container but misses another later in the query-processing code.

The deterministic adapter in Equation 2 reaches the same omission while retaining the public program’s lazy User and nested annotation. It produces a named-tuple point range and an ordinary-tuple neighbor. Retrieving the input relation from the graph lets the adapter reconstruct the same check used in this repair comparison.

A matched comparison tests the added condition and feedback. Two new agents start from the original unrepaired code with identical history, current scalar program, its observed failure, a callable execution tool, and instructions to run the supplied conditions before editing and submission. The second agent also receives the two composed programs and their failures. Both repairs pass official verification (Table 5). The repair given only current feedback fails both historical examples and the withheld named-tuple/lazy-User query; the repair given joint checks passes those behaviors and preserves the ordinary subclasses. The agent given joint checks runs the joint input before editing and again after removing a global conversion that broke an existing test. Its remaining range-endpoint failure directs the final correction.

Separate checks recover history but miss the interaction. A third agent receives the same history and current-program feedback, plus standalone named-tuple and ordinary-tuple checks using their original objects. Check count, feedback rounds, execution instructions, and repair budget match the composed arm. This repair passes official verification, both historical examples, and both ordinary-subclass checks, but only 1/3 joint queries and 2/3 mechanical checks; composed conditions yield 3/3 in each group (Table 5).

Compose roles to expose a broader requirement. The timezone relation provides another example. A fixed adapter varies the historical example’s three zones across database, active application, and destination roles and shifts the input by 182 days. Of 72 combinations, the source patch changes 20 from fail to pass; 20 pass on both versions and 32 fail on both. Applying the family to the four timezone content interventions shows that Requirement and Both pass all 72. Implementation passes the 40 source-supported combinations but fails the remaining 32, including 16 passed without memory. Role composition exposes how the current source-timezone requirement interacts with the historical destination requirement. A complete repair can therefore satisfy combinations beyond those corrected by the historical implementation.

E.3 EXECUTION FEEDBACK: GUIDING REQUIREMENT-BASED REPAIR

The addition can guide further repair. Two fresh agents start from the same completed ordinary timezone patch. Issue-based additional review returns it unchanged; supplying a generated historical check and its failed observations leads the other agent to use `self.get_tzname()` in both truncation methods. Both revisions pass official verification, but only the latter passes the historical and joint inputs. The intervention supplies corrective information that further issue-based review does not recover in this comparison.

Execution feedback makes an omitted condition concrete. Two fresh agents refine the same original failed Requirement candidate, both receiving its requirement again. One also receives the unchanged source-generated check and its actual failures for a field with a primary key or `unique=True`. Requirement-only review excludes the primary key but still fails the ordinary unique-field condition. Feedback adds generated-name disambiguation and passes both conditions. Both revisions pass official verification and the withheld joint sequence. The added behavior is recovery of the ordinary unique-field condition identified by feedback.

Feedback from the composed query. The Django12663 example also provides a refinement comparison. Four fresh refinements of its history-informed candidate illustrate the use of feedback: history alone yields one of two repairs passing all seven measured conditions; adding the constructed joint program and its failed observation yields two of two. All four pass official verification. Both agents given joint feedback execute the joint input and repair the later container reconstruction.

F BEHAVIORAL ANALYSIS WITH EXECUTABLE CONDITIONS

The development studies augment retrieved guidance with task-specific programs to examine whether a repair preserves the intended behavior. Figures 2 and 4 show their comparison and workflow in the main text; this appendix gives the construction protocol.

Values alone do not describe a condition. In Python, a named tuple is a tuple whose fields also have names. In our examples, constructing it takes two arguments; an ordinary tuple subclass takes one iterable. Both can hold the same values but exercise different code. Likewise, a timezone can describe the stored data or the requested output. Memory must preserve these roles, not just the values.

Two passing examples can hide a failing combination. Our Django case uses a lazy User object: a wrapper that loads the user when needed. A query using this object passes, as does an older range query using a named tuple to hold its endpoints. Putting the lazy object inside the named-tuple range in the current query fails. This motivates *condition composition*: inserting a historical trigger into the current operation (Figure 2).

F.1 EXECUTABLE-CONDITION STUDY PROTOCOL

The executable-analysis extension connects example construction to a repair loop (Figure 4). Construction records what a past repair taught and which conditions make that lesson relevant. During repair, retrieval selects useful experience, composition brings its conditions into the current workflow, and execution turns them into feedback for the next edit. The graph preserves the links between these stages, so the agent can trace a retrieved requirement to both its supporting example and its expected behavior.

F.2 REQUIREMENT MEMORY: PRESERVE TRIGGERING CONDITIONS

For these studies, an analysis record augments a retrieved requirement with a test example and its results before and after the historical repair:

$$m = (r, W, E^-, E^+), \quad W = (I, A, O), \quad (1)$$

where r states the requirement, I constructs its triggering input, A performs the operation, and O observes the expected outcome. We call W a witness: an executable example that fails on the historical code V^- and passes on its repaired version V^+ . E^- and E^+ record these executions.

A language-model writer extracts the requirement and example from the historical issue, patch, and relevant code. The example is checked against the historical repair, then stored with its requirement and execution results. The graph also records how inputs participate in an operation: which values form a range, for example, or which timezone specifies the output. These relations preserve the information needed to adapt an experience after retrieval.

Retrieve the requirement together with its context. The study retrieves relevant historical repairs, then follows record links to return their requirements, programs, execution results, and input roles together. A useful source may concern a related operation: a fix for deleting one kind of database constraint can help repair deletion of another kind. Keeping the requirement connected to its context gives the agent both a lesson and the information needed to apply it. Our implementation uses similarity search; Appendix G gives the retrieval settings.

F.3 CONDITION COMPOSITION: CHECK INTERACTIONS

Composition turns retrieved experience into a check for the current task. It starts with P_q , a runnable program that reproduces the current issue, and a historical requirement applicable to the current code. An *adapter*, a small program-transformation routine, uses the stored input relations to insert the historical trigger into P_q . The check keeps the current workflow and tests the expected behavior under this added condition.

For example, a historical range type can be introduced into the current query while preserving its input objects and query structure (Figure 2). Running this combination exposes interactions that neither the original query nor the historical example exercises alone. The same design applies to timezone conversion: the historical destination requirement is checked together with the current source-timezone behavior. Appendix G gives the input preparation and adapter constructions.

F.4 EXECUTION FEEDBACK: GUIDE REQUIREMENT-BASED REPAIR

The agent runs the constructed checks and uses failures to revise the patch. Each failure identifies an input and expected behavior that the repair needs to satisfy. Rerunning the same check gives successive edits a concrete repair objective.

G EXECUTABLE-ANALYSIS IMPLEMENTATION AND EVALUATION DETAILS

This appendix specifies the runnable-example and condition-composition extension used for behavioral analysis, introduced in Appendix F. Its programs are constructed for those studies; Section 3 defines the abstract experience memory and its use in planning and error recovery.

G.1 CONSTRUCTING HISTORICAL EXAMPLES

A language-model writer reads the historical issue, patch, and relevant code to propose a requirement and runnable example. Execution on the historical versions and researcher review check that the example exercises the intended behavior. Accepted examples are stored with their requirement, input roles, program, and before/after execution results.

G.2 RETRIEVAL AND REPAIR SETTINGS

The implemented retrieval path selects three historical repairs from the same repository using Qwen3-Embedding-8B vectors and FAISS (Zhang et al., 2025). Each has a validated example. It returns complete records and input relations, including types and argument roles.

The experiment configuration records source and target IDs, model and agent versions, index and adapter versions, prompts, budgets, and verifiers. This makes the retrieval, behavioral, and cost comparisons reproducible.

G.3 PREPARING THE CURRENT EXECUTABLE PROGRAM

The development range example extracts model and test snippets from the public issue and adds researcher-written setup and observation code. The range and timezone adapters are also researcher-written. The case inputs, expected behaviors, and control conditions are given in Appendix E.

G.4 COMPOSITION INTERFACE

The composer is the procedure that builds a combined check. It takes the current program P_q , memory m , input bindings, and available adapters. A binding identifies which object plays each role, together with its type, constructor, argument position, and code location. The composer returns the check, code that inspects its outcome (the observer), the mapping used, and whether construction succeeded. The steps are:

1. Match an adapter to the current operation and its type requirements.
2. Map current objects to historical input roles, preserving both triggers and the current operation sequence.
3. Construct the program and check its fixture, bindings, and observer. The expected outcome comes from the requirement, independently of the candidate patch.
4. Execute the check and return a behavioral failure as repair feedback. Record setup or observation errors separately so they can be distinguished from failures of the requirement.

The current adapters cover two Django operation families. The range adapter transforms the public query’s syntax tree, changing an equality lookup for x into a range with both endpoints equal to x , using the historical named-tuple type R :

$$P_q(x) \mapsto P_q^{\text{range}}(R(x, x)). \quad (2)$$

When equality and this one-point range select the same normalized, non-null value, the expected rows stay unchanged. The adapter preserves the lazy User and nested query; replacing that object with a lazy integer would lose the interaction. An ordinary-tuple range checks preservation of existing behavior. The timezone adapter constructs combinations of database, active application, and destination roles. Extraction code recovers the bindings these adapters support. Adding an operation family requires another adapter and its development tests.

H PUBLISHED SWE-CONTEXTBENCH REFERENCE RESULTS

Table 17 reproduces the resolved rates from Table 4 of Zhu et al. (2026), version 3. Supermemory leads the automatic memory settings reported there; oracle summaries use known task relationships to supply the relevant history. These published results provide context for our Related-Lite experiments. Direct comparisons require matching task IDs, backbone, agent, and attempt budget.

Table 17: Published SWE-ContextBench Lite results: 99 related tasks, Claude Sonnet 4.5. Rates are copied as reported, including OpenViking’s 29.20%. Δ is the percentage-point change from no context. Oracle rows are separated from automatic retrieval.

Setting	History access	Resolved (%)	Δ (pp)
No context	None	26.26	0.00
Free context	Agent-selected trajectories	26.26	0.00
Free summary	Agent-selected summaries	22.22	−4.04
Mem0	Automatic retrieval	24.24	−2.02
OpenViking	Automatic retrieval	29.20	+2.94
Supermemory	Automatic retrieval	30.30	+4.04
Oracle context	Supplied related trajectory	27.27	+1.01
Oracle summary	Supplied related summary	34.34	+8.08

On the separate 376-task set, Table 3 reports its best resolution rate of 23.94% for GPT-5.3 Codex with oracle summaries. Supermemory’s 55.95% in Table 4 is a test-level FAIL_TO_PASS rate, not task resolution. LangMem has retrieval-quality results in Table 5 but no resolved rate in Table 4.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing, including drafting and revising the abstract.

Table 18: Recorded repair resolution rates (%). Panel A gives our three-benchmark comparison. Panel B groups published reference results by their reported configuration. Bold marks the best result per benchmark in Panel A. An em dash indicates that the cited source does not report the corresponding benchmark.

A. Three-benchmark comparison

Method	Verified500	Related-Lite99	DeepSWE113
No memory	61.6	21.4	60.8
FAISS Flat	65.2	26.8	64.6
ExpeL	62.4	20.7	58.7
Agent KB	64.0	38.2	61.5
ExpeRepair [†]	63.6	31.3	64.4
ReasoningBank	67.0	24.9	61.7
ACE [†]	62.2	28.1	62.8
Supermemory	63.6	26.5	66.2
ContextGraph	69.2	36.3	72.1

Verified500, Related-Lite99, and DeepSWE113 use DeepSeek V4 Pro 0813 and mini-SWE-agent. Related-Lite99 and DeepSWE113 rates are means across five runs. [†] ExpeRepair and ACE are our adaptations to mini-SWE-agent. Method sources: ExpeL (Zhao et al., 2024), Agent KB (Tang et al., 2025), ExpeRepair (Mu et al., 2026), ReasoningBank (Ouyang et al., 2026), ACE (Zhang et al., 2026b), and Supermemory (Daga et al., 2026).

B. Published reference results, grouped by their reported configuration

Method	Backbone	Verified500 (%)	Related-Lite99 (%)	Source
ExpeRepair	Claude 3.5 Sonnet	57.2	—	ER
ExpeRepair	Claude 4 Sonnet	74.6	—	ER
ReasoningBank	Gemini 2.5 Flash	38.8	—	RB
ReasoningBank	Gemini 2.5 Pro	57.4	—	RB
No memory	Claude Sonnet 4.5	—	26.26	SC
Supermemory	Claude Sonnet 4.5	—	30.30	SC
Oracle summary	Claude Sonnet 4.5	—	34.34	SC

ER: ExpeRepair, Table 1; RB: ReasoningBank, Table 2; SC: SWE-ContextBench v3, Table 4. ER uses its own repair pipeline with occasional o4-mini fallback; RB uses mini-SWE-agent; SC uses Claude Code. Oracle summaries supply the known related history. Published scores use their original budgets and memory settings.